

COMP 8005

Assignment 4

Design

Manraj Bhullar

A01018465

March 4, 2026

Overview

The purpose of this design document is to describe the next and final iteration of the distributed password cracking system. In the previous version, the system gained the ability to support multiple workers where several machines worked together to search the password space. Workers repeatedly requested chunks of work from the controller and processed those chunks while the controller handled worker registration, job assignment, and stopping the entire system once the password was found.

In this version, the system adds checkpointing and failure recovery so that progress is not lost if a worker crashes. Workers periodically send checkpoint updates stating how far they have progressed within their assigned chunk. The controller uses this progress information to track how much of each chunk has already been completed, allowing it to reassign only the remaining portion of the work from the last known progress point of the worker. This allows the system to avoid repeating work that has already been completed while still recovering any unfinished portion of the search space if a worker fails.

The system also supports manual interruption such as Ctrl+C, allowing a job on a worker to be stopped and then continued by reassigning it from that point. When triggered, the worker sends its final progress before exiting, and the controller uses the same reassignment logic to continue the remaining work when assigning the next job.

Reassignment Scenarios:

- Worker declared dead after exceeding heartbeat miss threshold
- Controller detects worker connection closed unexpectedly
- Manual interruption such as Ctrl+C

Goal

Controller

The goal of the controller is to coordinate a distributed password cracking operation across multiple workers. It must parse the shadow entry, divide the global search space into chunks, assign work dynamically, monitor worker heartbeats, coordinate checkpoint reporting, and enforce a global stop condition once the password is found.

Worker

The goal of the worker is to repeatedly request work from the controller and crack only its assigned portion of the search space using multiple threads. The worker must also periodically send checkpoint updates stating its progress and maintain a local checkpoint so that work can be recovered or resumed if the worker crashes.

Success

Controller

The controller is considered successful when it correctly registers multiple workers, distributes unique chunks of work without duplication, and stops all workers once a valid password is discovered. It must also track worker progress through checkpoint reports and ensure that unfinished work can still be completed if a worker fails. Finally, It must report final timing measurements and performance statistics.

Worker

The worker is successful when it correctly requests chunks, attempts to crack each without duplication, responds to heartbeats, reports results, and stops immediately when any worker finds the correct password. The worker must also send checkpoint reports at the configured interval and maintain a local checkpoint so that if it restarts after a crash, it can resume progress when possible.

Invariant

Controller

The controller must never assign the same chunk twice. Each chunk must be issued at most once unless a worker fails and the unfinished portion must be reassigned.

Checkpoint information must accurately track worker progress so that completed work is not repeated after recovery. Once the password is found, no additional chunks may be assigned and all workers must be instructed to stop.

Worker

The worker must never attempt candidates outside its assigned chunk. Each candidate inside the chunk must be tested exactly once across all threads. Checkpoint reporting and local checkpoints must also accurately reflect the worker's progress so that completed work is not repeated.

Responsibilities

Controller

1. Accepting a shadow file, username, port, heartbeat interval, chunk size and checkpoint interval from the command line.
2. Parsing the shadow file to extract the hashing algorithm identifier, salt, and password hash.
3. Accepting multiple worker registrations.
4. Maintaining a global chunk allocator for the search space.
5. Responding to worker chunk requests with unique disjoint ranges.
6. Receiving periodic heartbeat responses from workers.
7. Receiving checkpoint reports from workers.

8. Ensuring unfinished work is reassigned if a worker fails.
9. Receiving completion notifications and final results.
10. Enforcing a global stop when any worker finds the password.
11. Reporting timing breakdowns and final results.

Worker

1. Register with the controller.
2. Request a chunk of work.
3. Crack assigned chunk using T threads.
4. Report FOUND immediately if discovered.
5. Request next chunk if exhausted.
6. Respond to heartbeat requests.
7. Send checkpoint reports to the controller.
8. Maintain a local checkpoint.
9. Resume from the local checkpoint if the last chunk has not been reassigned.
10. Exit when STOP received.

Testing Strategy

Controller

Test 1: Worker Checkpoint Reporting

Steps

1. Run the controller and a worker with a set checkpoint interval.
2. Allow the worker to process work long enough for several checkpoint intervals.

Expected Result

- Worker sends checkpoint reports to the controller every checkpoint interval.

What it Proves

- Worker correctly reports checkpoint progress and the controller is aware how much of the chunk remains.

Test 2: Worker Failure Reassignment

Steps

1. Run the system with multiple workers.
2. Kill a worker while it is still working on a chunk.

Expected Result

- Controller detects worker is lost, checks its last reported progress, and gives remaining chunk to the next available worker.

What it Proves

- Controller correctly handles crash recovery by reassigning unfinished work.

Test 3: No Duplicate Work after Recovery

Steps

1. Run the controller and allow a worker to send at least one checkpoint.
2. Kill the worker and let another worker finish the remaining work of that chunk.

Expected Result

- Work resumes from the last checkpoint and no candidate tested prior to failure is repeated.

What it Proves

- Controller correctly uses checkpoint progress to avoid repeating work.

Worker

Test 1: Checkpoint Reporting

Steps

1. Run a worker and let it crack a chunk long enough to hit multiple checkpoint intervals.

Expected Result

- Worker sends checkpoint reports showing its progress within the current chunk every interval.

What it Proves

- Worker correctly generates and sends checkpoints during jobs.

Test 2: Local Checkpoint Resume

Steps

1. Let a worker process part of a chunk until a local checkpoint exists.
2. Terminate and restart the worker.

Expected Result

- Worker resumes from its local checkpoint instead of getting a brand new chunk from the controller unless it has already been reassigned.

What it Proves

- Worker can restart and resume progress using its own local checkpoint and not duplicate work for that chunk.

FSM

State Meanings

Controller

State	Meaning
START	Program is ran.
PARSE_ARGS	Command-line arguments get parsed.
HANDLE_ARGS	Parsed arguments are validated and prepared for use.
PARSE_SHADOW	Shadow file is read and the username's entry is parsed to extract algorithm, salt, and hash.
LISTEN	Opens connection socket and continually accepts new worker connections.
REGISTER_WORKER	A worker joins the distributed cracking session.
MONITOR_PROGRESS	Tracks all active workers and receives updates such as heartbeats or checkpoints.
DISPATCH_JOB	Assigns a portion of the search space to a worker requesting a new chunk.
STOP_ALL	Stops all workers after the password is found.
REPORT_FOUND	Outputs final cracking results.
ERROR	Reports when a state error occurs and redirects to cleanup.
CLEANUP	Program releases resources and closes connections.
USAGE	Displays help message when program input is invalid.
EXIT	Program exits.

Worker

State	Meaning
START	Program is ran.
PARSE_ARGS	Command-line arguments get parsed.
HANDLE_ARGS	Parsed arguments are validated and prepared for use.
REGISTER	Worker connects and registers with the controller.
REQUEST_JOB	Worker requests next job parameters from controller.
CRACK	Cracking process for currently assigned job/chunk while responding to heartbeat requests and providing checkpoint updates.
FORCE_STOP	Worker stops if the password has been found by another worker.
SEND_JOB_RESULT	Worker sends results from current cracking job that has finished.
ERROR	Reports when a state error occurs and redirects to cleanup.
CLEANUP	Program releases resources and closes connections.
USAGE	Displays help message when program input is invalid.
EXIT	Program exits.

State Transition Tables

Controller

Current State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS

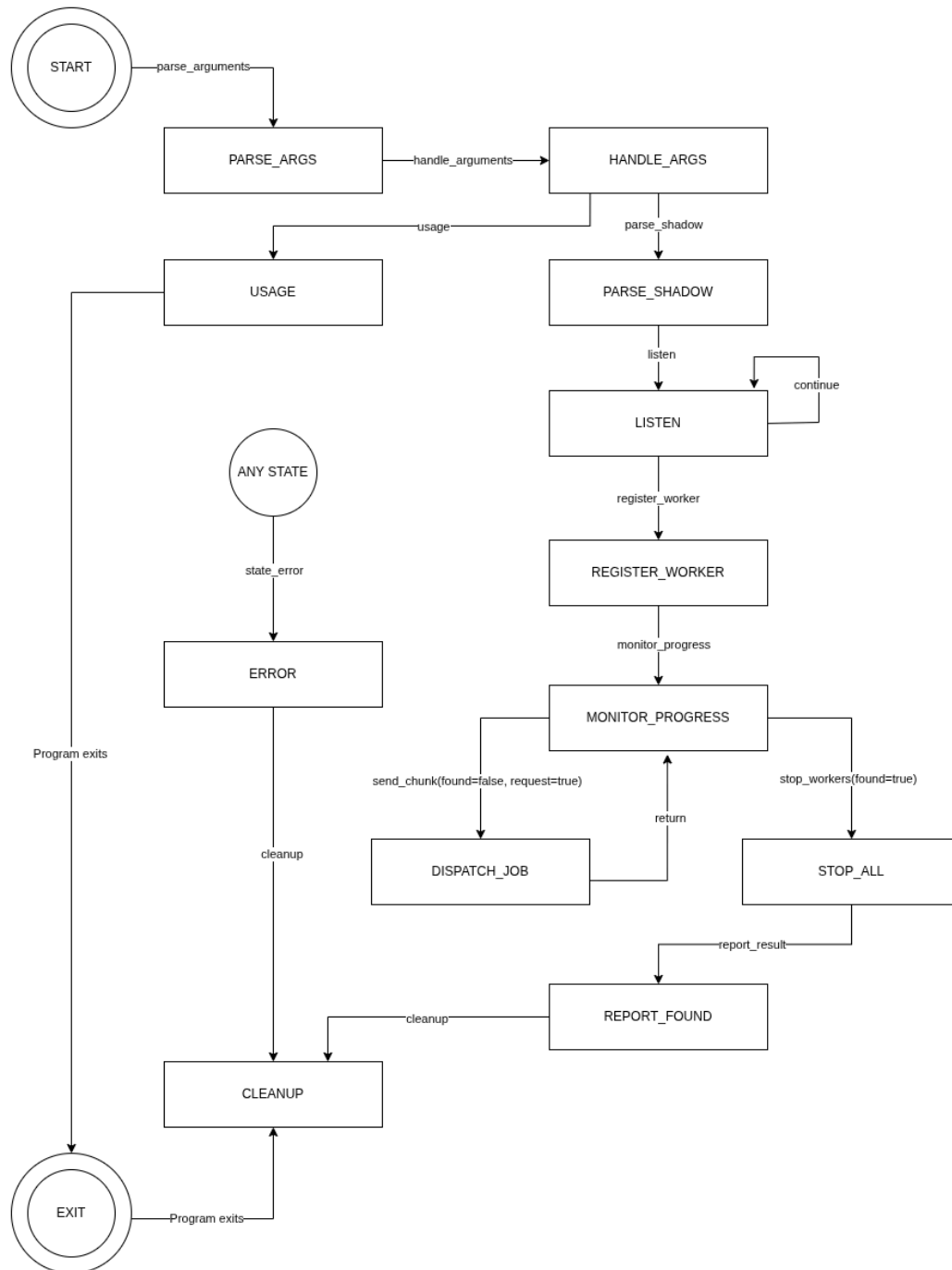
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
HANDLE_ARGS	parse_shadow	USAGE	PARSE_SHADOW
PARSE_SHADOW	listen	ERROR	LISTEN
LISTEN	register_worker	ERROR	MONITOR_PROGRESS
MONITOR_PROGRESS	send_chunk	ERROR	DISPATCH_JOB
	stop_workers	ERROR	STOP_ALL
STOP_ALL	report_result	ERROR	REPORT_FOUND
REPORT_FOUND	cleanup	ERROR	CLEANUP
ERROR	cleanup		CLEANUP
CLEANUP	Program exits		EXIT
USAGE	Program exits		EXIT

Worker

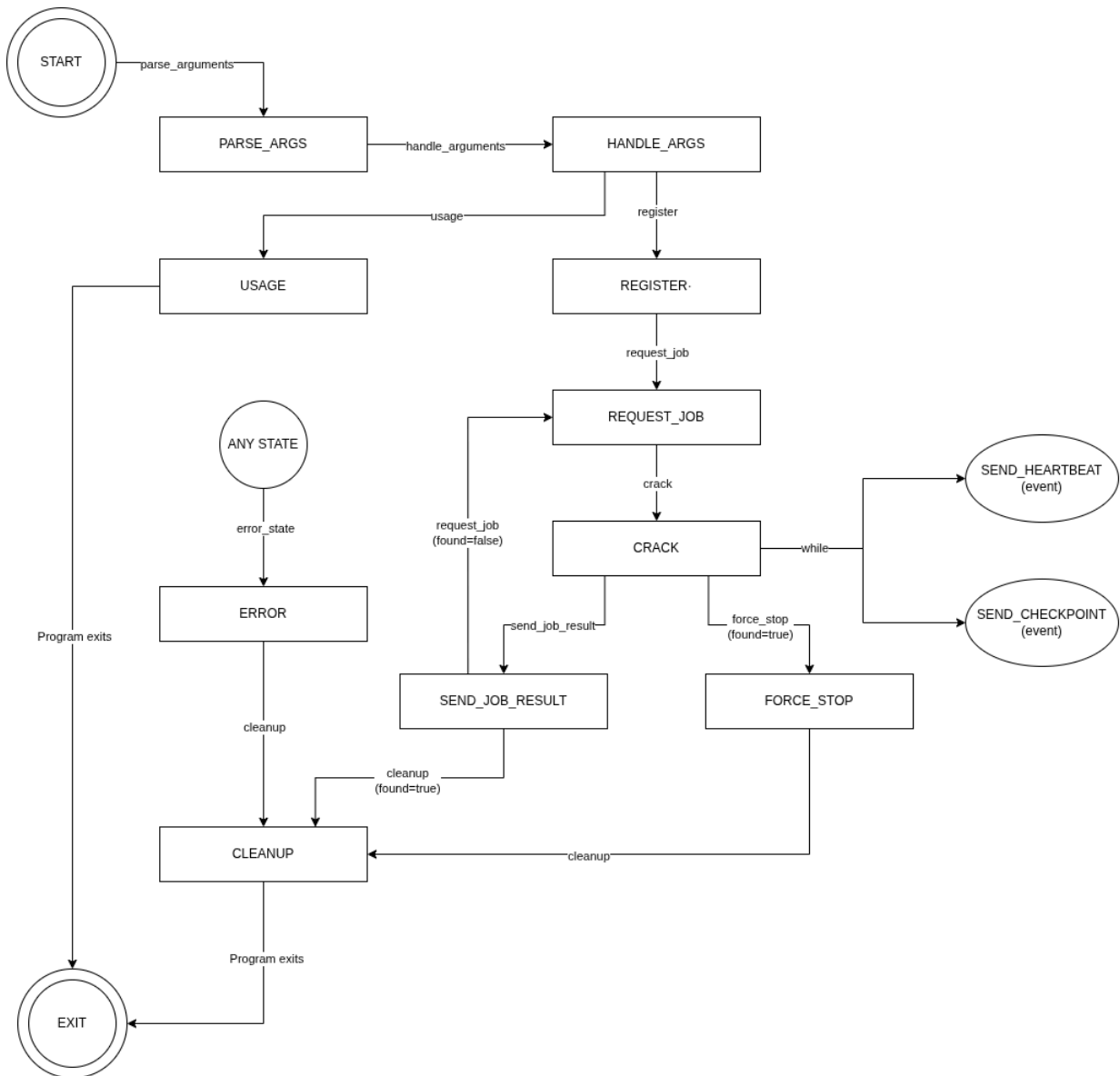
Current State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
HANDLE_ARGS	register	USAGE	REGISTER
REGISTER	request_job	ERROR	REQUEST_JOB
REQUEST_JOB	crack	ERROR	CRACK
CRACK	force_stop	ERROR	FORCE_STOP
	send_job_result	ERROR	SEND_JOB_RESULT
FORCE_STOP	cleanup	ERROR	CLEANUP
SEND_JOB_RESULT	request_job	ERROR	REQUEST_JOB
	cleanup	ERROR	CLEANUP
ERROR	error_state		CLEANUP
CLEANUP	Program exits		EXIT
USAGE	Program exits		EXIT

FSM Diagrams

Controller



Worker



Functions

Controller

Name	Inputs	Outputs	Responsibility
------	--------	---------	----------------

parse_arguments	Command-line inputs	Parsed arguments	Parse command-line arguments for shadow file, username, port, heartbeat interval, and chunk size.
handle_arguments	Parsed arguments	Validated arguments	Validate arguments and ensure values are usable before program execution.
parse_shadow	Shadow file path and username	Password data	Read the shadow file and extract the selected user's password information.
listen	Port number	Controller socket	Initialize the controller server socket and accept worker connections.
register_worker	Worker connection	Registration status	Accept and record a worker registration so it can participate in cracking.
monitor_progress	Active worker connections	Worker progress information	Manage all active workers, receive heartbeat/checkpoint updates, and detect chunk completion or password discovery.
send_chunk	Worker request and global allocator	Assigned job	Assign the next available portion of

			the search space to a requesting worker.
stop_workers	Found password flag	Stop flag set	Notify all workers to terminate once the password has been discovered.
report_result	Final cracking result	Printed cracking results	Display final result and timing breakdown.
state_error	Error context	Error message	Handle runtime state errors and transition to cleanup.
cleanup	Error message	Exit status	Close sockets and terminate the controller cleanly.
usage	Program context	Usage message	Print usage instructions when arguments are invalid.

Worker

Name	Inputs	Outputs	Responsibility
-------------	---------------	----------------	-----------------------

parse_arguments	Command-line inputs	Parsed arguments	Parse command-line arguments for shadow file, username, port, heartbeat interval, and chunk size.
handle_arguments	Parsed arguments	Validated arguments	Validate arguments and ensure values are usable before program execution.
register	Connection details	Registration status	Connect to the controller and register the worker.
request_job	Controller connection	Assigned job details	Request a new portion of the search space from the controller.
crack	Assigned job details and thread count	Progress updates and job results	Process cracking job while responding to heartbeat requests and sending checkpoint updates.
force_stop	Stop signal from controller	Stop confirmation	Stop current job if another worker finds password.
send_job_result	Job results data	Result sent status	Send the discovered password or exhaustion status to the controller.
error_state	Error context	Error message	Handle runtime state errors and transition to cleanup.

cleanup	Error message	Exit status	Close sockets and terminate the controller cleanly.
usage	Program context	Usage message	Print usage instructions when arguments are invalid.